

Milestone 3: Critical Evaluation and Transfer

Priority Inheritance Protocol (Resource Access Protocols)

Nnaemeka Nnachi-Egwu
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
nnaemeka.nnachi-egwu@stud.hshl.de

I. CONCRETE STRENGTHS (WITH BUTTAZZO CITATIONS)

S1 — Bounded priority inversion (Theorem 7.2, p. 219):

Without any protocol, the inversion window is unbounded — any number of medium-priority tasks can prolong it indefinitely (Fig. 7.4). PIP bounds blocking to at most $\alpha_i = \min(l_i, s_i)$ critical sections per task. This is a *formal guarantee*, not a heuristic.

S2 — Transparency (Buttazzo §7.10, p. 247, Table 7.5):

`pi_wait/pi_signal` are drop-in replacements for standard semaphore primitives. Task *application code* does not change. No ceiling declarations required. This is the only protocol (besides NPP) rated “transparent.” *Practical consequence*: POSIX `PTHREAD_PRIO_INHERIT` converts any existing mutex-protected code to PIP with one attribute call. Mars Pathfinder fix: one line of configuration.

S3 — Low pessimism vs. HLP (Buttazzo p. 214):

HLP blocks at activation time based on ceiling of *entered* CS, even if the resource-accessing branch is never taken at runtime. PIP defers to actual *wait*: no blocking unless genuine resource contention. *In practice*: PIP’s observed blocking is usually below the B_i bound (the bound is worst-case, not average-case).

S4 — Transitive inheritance for nested CS (Lemma 7.2, p. 218):

For nested critical sections, priority propagates transitively up the chain, preventing intermediate holders from being preempted. Without this, a 3-level nesting would still suffer priority inversion at the intermediate level.

S5 — Direct feed into schedulability (Buttazzo §7.9):

B_i values (Eqs. 7.9–7.11) plug directly into Liu-Layland (Eq. 7.19) and RTA (Eq. 7.22) without modification to the scheduling framework. PIP is the only protocol that provides this tight integration with standard fixed-priority analysis.

II. CONCRETE LIMITATIONS AND COUNTEREXAMPLES

A. L1: No Deadlock Prevention (Buttazzo §7.6.5, Fig. 7.13, p. 226)

Minimal counterexample: Two tasks (τ_1 high, τ_2 low), two semaphores (S_a, S_b):

- τ_1 : `wait( $S_a$ ); wait( $S_b$ ); ...`
- τ_2 : `wait( $S_b$ ); wait( $S_a$ ); ...`

Timeline: t_1 : τ_2 locks S_b ; t_2 : τ_1 preempts; t_3 : τ_1 locks S_a ; t_4 : τ_1 tries $S_b \rightarrow$ blocked, τ_2 inherits P_1 ; t_5 : τ_2 tries $S_a \rightarrow$ **deadlock**.

Why PIP cannot prevent it: `pi_wait` only boosts priority; it never denies a lock on a free semaphore. The deadlock arises from a lock-order cycle, which PIP has no mechanism to detect.

Buttazzo (p. 227): “The deadlock does not depend on the Protocol but is caused by an erroneous use of semaphores.”

Fix: Total ordering on semaphore acquisition (by ID) eliminates the cycle. Or replace PIP with PCP (Theorem 7.3: PCP prevents deadlock).

Note: PIP does not *cause* deadlocks — it is neutral. The same deadlock would occur without any protocol.

B. L2: No Chained Blocking Prevention (Buttazzo §7.6.5, Fig. 7.12, p. 225)

Minimal counterexample: $\tau_1 \succ \tau_2 \succ \tau_3$; two semaphores S_a, S_b ; τ_2 holds S_b ; τ_3 holds S_a . τ_1 arrives: blocked on S_a (first blocking), then unblocked, then blocked on S_b (second blocking) $\rightarrow$ **two** blockings.

With n distinct semaphores, each held by a different lower-priority task: τ_1 blocked up to $\alpha_1 = \min(n, n) = n$ times. PCP (Theorem 7.4) limits this to exactly 1.

C. L3: Kernel Implementation Complexity (Table 7.5, p. 248)

PIP is the *only* protocol rated “Hard” in Table 7.5. Root cause: transitive chain traversal in `pi_wait` must be executed atomically in the kernel. Each step must: (1) read current holder’s blocked-on semaphore, (2) boost that holder, (3) repeat if that holder is also blocked.

This loop adds non-deterministic interrupt latency. Production kernels (Linux `rtmutex.c`) bound chain depth to prevent infinite loops — which means formal guarantees may not fully hold on the implementation.

Transparency (S2 above) is for *application* programmers; the *kernel* implementation is decidedly non-trivial.

Note on WCET sensitivity: B_i formulas (Eqs. 7.9–7.11) require exact $\delta_{j,k}$ values. Abella et al. [WCET] show no current WCET method is fully trustworthy — especially on modern hardware with caches, pipelines, and multicore interference. Two distinct pessimism sources compound: (1) structural: B_i^l sums CS that cannot simultaneously block (Lemma 7.4 pessimism), (2) input uncertainty: conservative WCET estimates inflate $\delta_{j,k}$, making B_i unnecessarily large. *This applies to all protocols with CS-duration-based blocking formulas, not PIP specifically.*

III. RUNTIME, MEMORY, AND SCALABILITY

- Offline blocking computation: $\mathcal{O}(mn^2)$ per Fig. 7.11 algorithm. Feasible for $n, m \leq \text{tens}$.
- Exact algorithm: $\mathcal{O}(2^n)$ — impractical for $n > 15$.
- Runtime (per `pi_wait`): chain traversal — worst case proportional to nesting depth; bounded by implementation limits.
- Additional TCB fields: active priority, blocked-on semaphore ID, semaphore wait queues. $\mathcal{O}(n + m)$ extra state.
- Semaphore queues: $\mathcal{O}(n)$ total across all semaphores (each task in at most one queue at a time).

IV. TRANSFER ANALYSIS

A. Where PIP Is Suitable

- 1) **Single-core RTOS with bounded nesting:** l_i and s_i are small; α_i is small; B_i is manageable. Total semaphore ordering enforced $\rightarrow$ no deadlock risk.
- 2) **POSIX legacy migration:** `PTHREAD_PRIO_INHERIT` is one attribute call; no code restructuring. Transparent, immediate improvement in worst-case timing.
- 3) **Automotive ECU (classic AUTOSAR, single-core):** Bounded blocking time for ASIL certification; short CS WCETs for low-complexity ECU code; RM scheduling standard.
- 4) **Any system where deadlock is provably absent by design:** If all semaphores acquired in fixed total order (common rule), PIP’s only remaining weakness is chained blocking (Theorem 7.2 bounds it), and the implementation overhead is acceptable.

B. Where PIP Is Unsuitable

- 1) **Multiprocessor/multicore systems:** Priority boost on a remote processor requires IPI; latency is non-deterministic. Gracioli et al. [GP]: IPI overhead has no equivalent in Buttazzo’s B_i formula $\rightarrow$ analysis invalid. Use MPCP, FMLP, or MSRP.
- 2) **EDF-scheduled systems:** PIP undefined for dynamic priorities. Use SRP [Bak91] which natively supports EDF via preemption levels.
- 3) **Safety-critical systems requiring certified deadlock freedom:** DO-178C, IEC 61508, ISO 26262 high-ASIL require provable deadlock absence. PIP alone is insufficient; use PCP or enforce and prove total semaphore ordering.
- 4) **Deep nested CS with many tasks:** Transitive chain in `pi_wait` adds atomic interrupt latency; α_i can be large; blocking bound grows.
- 5) **Systems with unreliable WCET estimates:** If $\delta_{j,k}$ cannot be bounded tightly (modern hardware, caches, multicore interference), the B_i computation is unsound. B_i under-estimates $\rightarrow$ false schedulability guarantee.

V. PROPOSED EXTENSIONS

E1 — POSIX validation: Implement the 3-task set with `pthread + SCHED_FIFO`, run under `PTHREAD_PRIO_INHERIT` vs. plain mutex. Measure actual τ_1 blocking with `clock_gettime`. Hypothesis: observed blocking $\leq B_1 = 4$ in all runs, and strictly less in most (bound is not tight).

E2 — UPPAAL deadlock witness: Add a second semaphore S_a to the UPPAAL model with reverse acquisition order in one task. Verify that `A[] not deadlock fails` and the counterexample trace matches Fig. 7.13, making the deadlock limitation formally witnessed.

E3 — Tight blocking computation: Implement Rajkumar’s exact $\mathcal{O}(2^n)$ algorithm in Python for $n \leq 8$. Compare against Eq. (7.11)’s result on the 4-task, 3-semaphore example (Buttazzo Table 7.1, p. 222) to quantify the pessimism gap.

E4 — Multiprocessor overhead model: Using the overhead taxonomy from Gracioli et al. [GP] (Table 1: Δ_{cxs} , Δ_{scd} , Δ_{ipi}), derive a modified B_i formula for a dual-core system with Global-EDF + PIP semantics and check schedulability under realistic IPI latency values from [GP].